



OBJECT ORIENTED ANALYSIS AND DESIGN

Ch - 10

Inheritance and Polymorphism

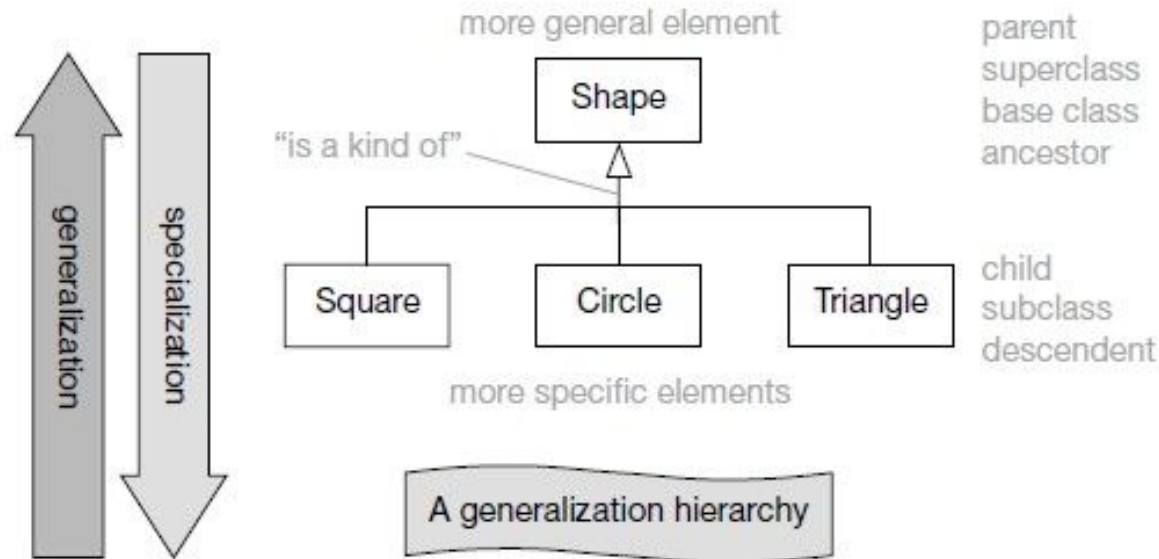
A study material for the students of GLS University

Generalization

- Generalization is the process of extracting common characteristics from two or more classes and combining them into a general (parent) class.
- The specific class (child) has all features of the general class but adds some more details.
- It shows a “is-a” relationship.

Example: Tree (general) → Oak Tree (specific)

Class generalization



Specialization (Top-Down):

Start with a broad/general concept (e.g., Shape).
Add details step by step (Circle, Square, Triangle).

Generalization (Bottom-Up):

Start with many specific classes (e.g., Car, Bus, Truck).
Notice common features (all are Vehicles) and create a general superclass.

Class Inheritance

When you model generalization in UML, you are also modeling inheritance in programming.

What gets inherited?

- Attributes (variables/data members)
- Operations (methods)
- Associations (relationships with other classes)
- Constraints (rules)

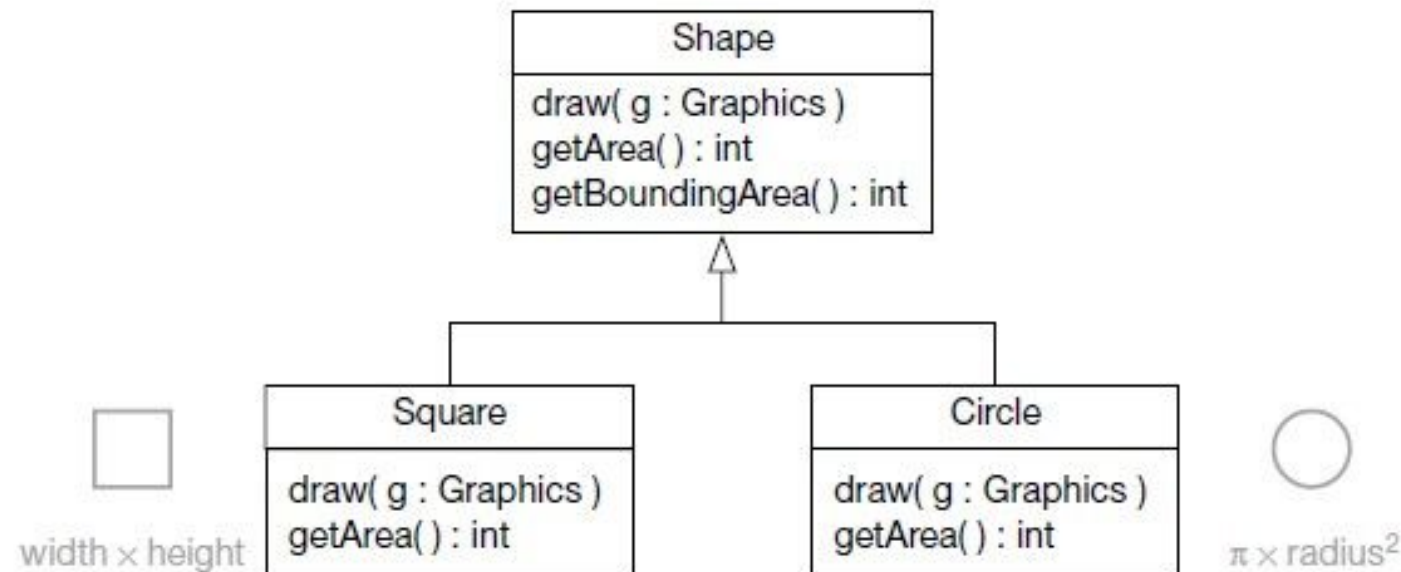
What subclasses can do?

- Reuse all features from the superclass.
- Add new attributes/methods of their own.
- Override superclass methods to provide their own implementation.

Overriding

Subclasses override inherited operations by providing a new operation with the same signature.

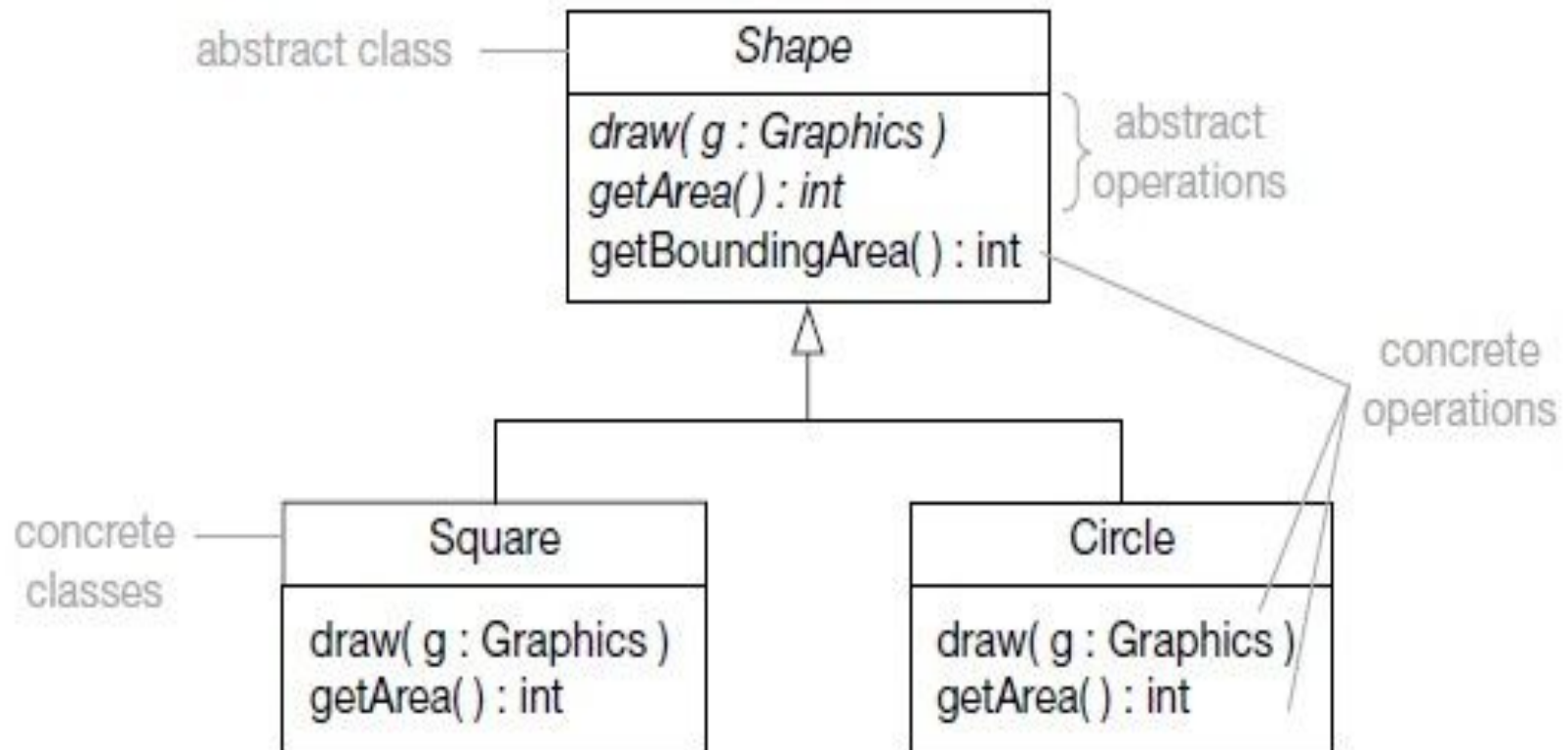
To override a superclass operation, a subclass must provide an operation with exactly the same signature as the superclass operation it wishes to override.



Abstract Classes and Abstract Operations

- No implementation
- Abstract classes have one or more abstract operations, and they can't be instantiated
- In UML, writing the operation name in italics
- Incomplete as operations don't have an implementation
- can't instantiate classes and they are therefore known as abstract classes
- You write the class name in italics to show that it is abstract

Example



Advantages of using abstract classes and operations

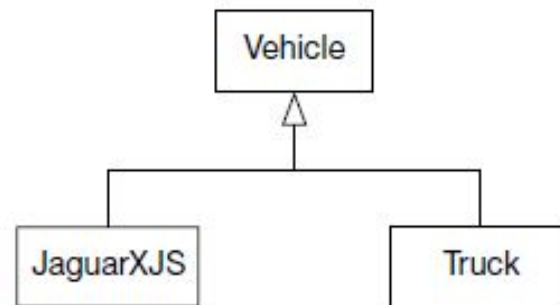
You can define a set of abstract operations in the abstract superclass that must be implemented by all subclasses. You can think of this as defining a “contract” that **all concrete subclasses must implement.**

You can write code to manipulate Shapes and then substitute Circle, Square and other Shape subclasses as appropriate. According to the substitutability principle, code written to manipulate **Shapes should also work for all Shape subclasses.**

Level of Abstraction

- At same level in a generalization hierarchy - should be at the same level of abstraction
- Levels of abstraction from the most general at the top to the most specific at the bottom
- Maintain a uniform level of abstraction at each level of the generalization hierarchy

Example:



- Clearly this is a lower level of abstraction than Truck. You can fix the model quite easily by introducing a Car superclass between JaguarXJS and Vehicle

Multiple Inheritance

Multiple Inheritance - A class can have more than one direct superclass

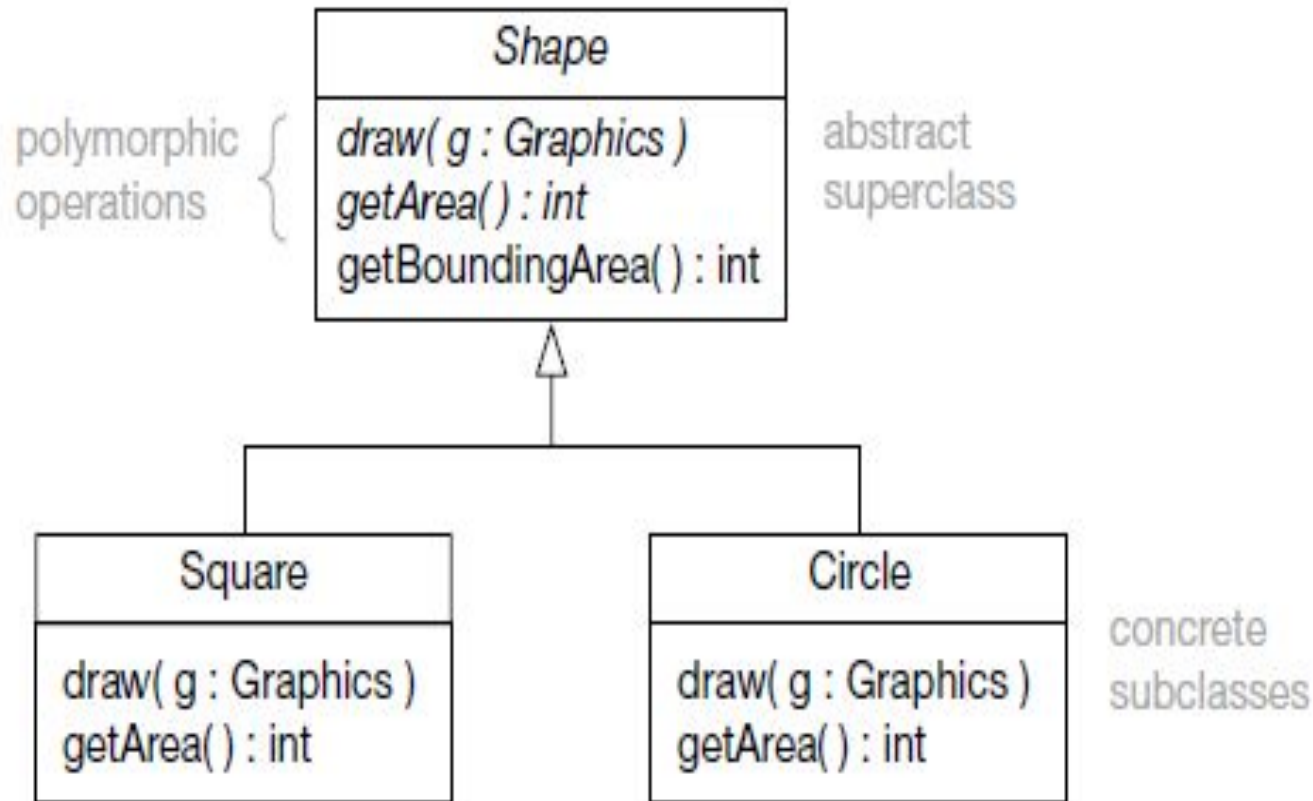
UML allows a class to have more than one direct class

The subclass inherits from all of its direct superclasses

Polymorphism

- Polymorphism means “many forms”
- A polymorphic operation is one that has many implementations
- You have already seen two polymorphic operations in the Shape example.
- The abstract operations `draw( )` and `getArea( )` in the Shape class have two different implementations – an implementation in the Square class and a different implementation in the Circle class. The operations have “many forms” and are therefore polymorphic

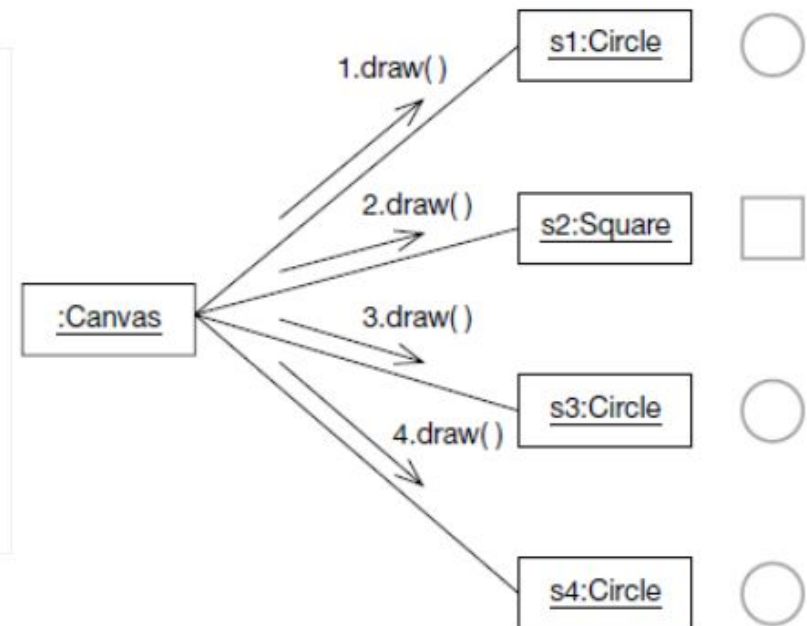
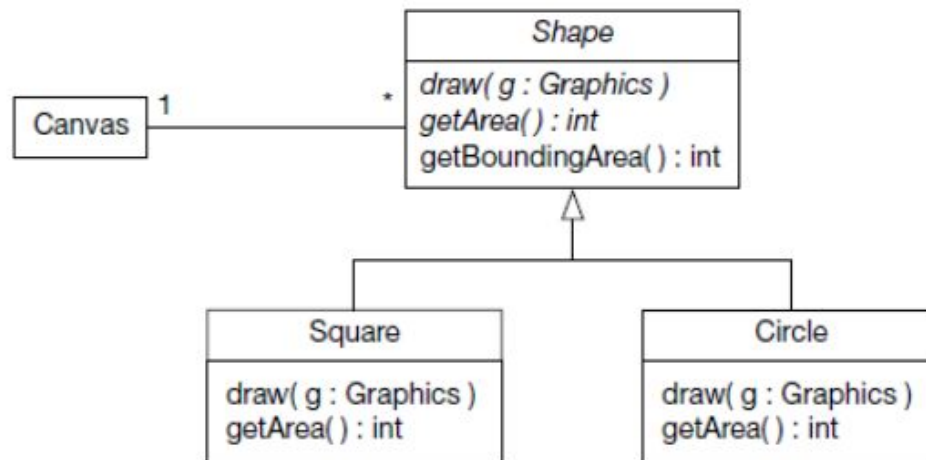
Example



Polymorphism – objects of different classes have operations with the same signature but different implementations

If you send objects of the Square class the message draw() they will draw a square, and if you send objects of the Circle class the same message they will draw a circle

Polymorphism Example



Advanced Generalization

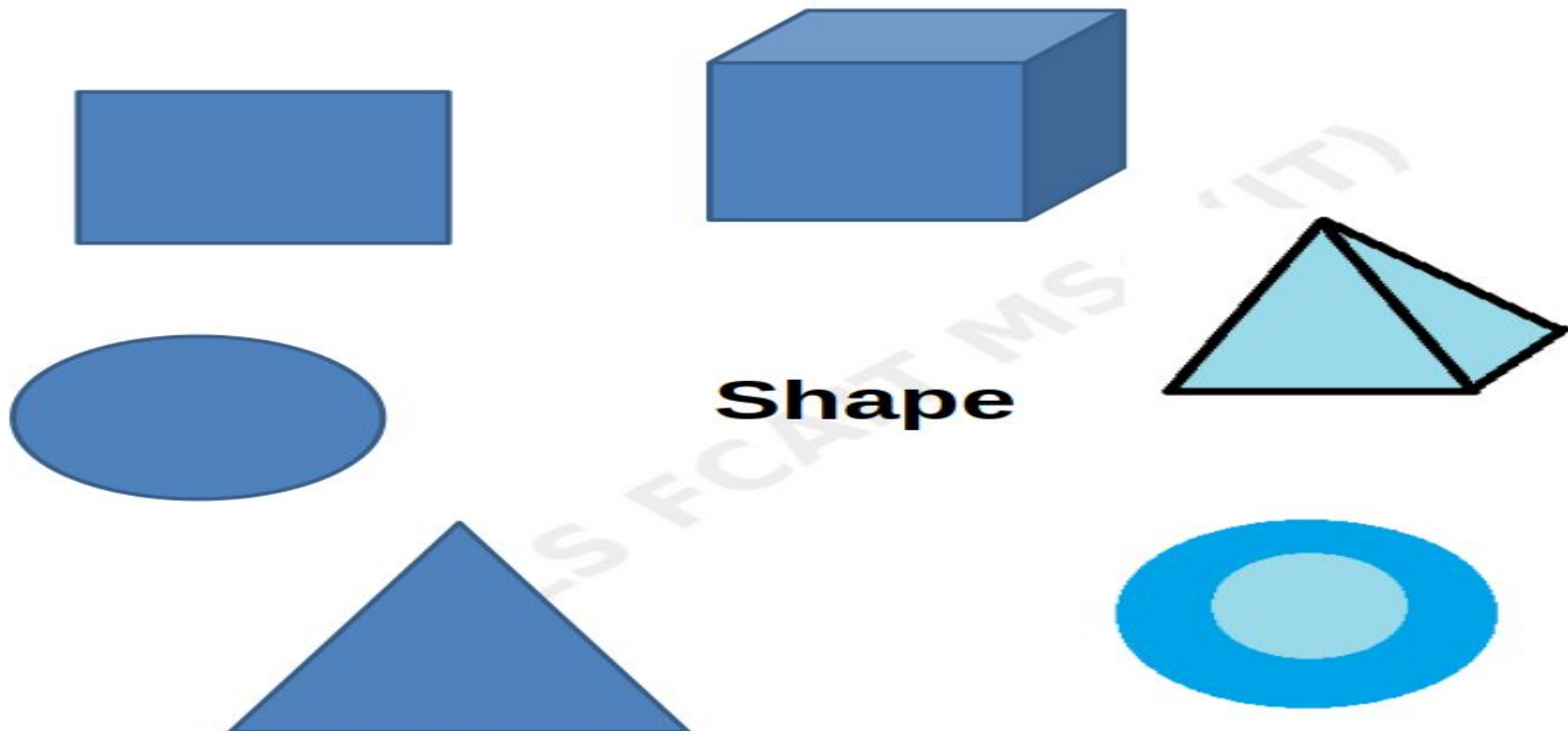
Two advanced aspect of generalization

- Generalization sets
- Powertypes

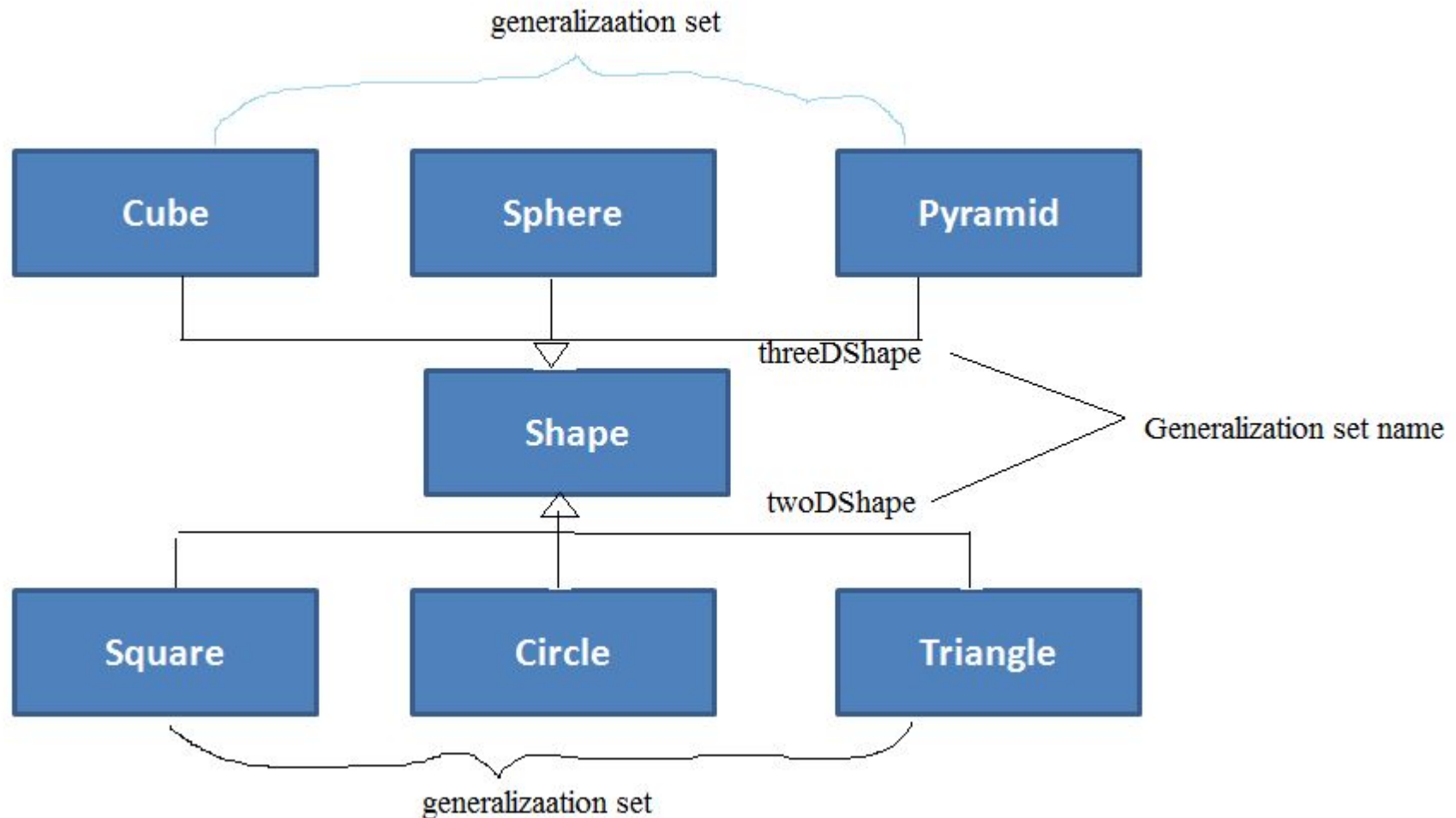
Generalization Sets

A generalization set is a collection of related generalizations that apply to the same parent class.

Figure illustrates that the superclass Shape has many subclasses. On examination of these subclasses you can see that there are two distinctly different groups of Shapes : two-dimensional shapes and three-dimensional shapes



You can indicate this partitioning of subclasses on a class diagram by associating each group of shapes with a different generalization set types



Generalization sets can have constraints applied to them

Constraints:

1. {complete}

The parent class is fully divided into its subclasses.

Example:

TrafficLight can only be Red, Yellow, or Green.

No object exists directly as TrafficLight, only as one of the subclasses.

2. {incomplete}

The parent class can exist on its own OR may have other unspecified subclasses in the future.

Example:

Vehicle may have subclasses Car, Bike, but there could still be Vehicle objects without being classified into any subclass.

Complete

Every superclass instance belongs to at least one subclass

All possibilities covered

Closed classification

Incomplete

Some superclass instances may belong to none

Not all possibilities covered

Open classification

3. {disjoint}:

An object can belong to only one subclass at a time.

Example:

1. Employee can be either FullTimeEmployee OR PartTimeEmployee, not both.
2. 2D shapes not belong to 3D shapes

4. {overlapping}

An object can belong to multiple subclasses at the same time.

Example:

Person can be a Student AND an Employee at the same time.

Disjoint

Instance can belong to only one subclass

Exclusive classification

Example: Vehicle → Car/Bus/Truck

Overlapping

Instance can belong to multiple subclasses

Multiple classification allowed

Example: Person → Student/Employee

Powertypes

A powertype is a classifier (class) whose instances are the subclasses of another classifier.

In other words:

We have a generalization set (a set of subclasses).

The powertype is a class that represents the types in that generalization set.

Normal class → has objects as instances.

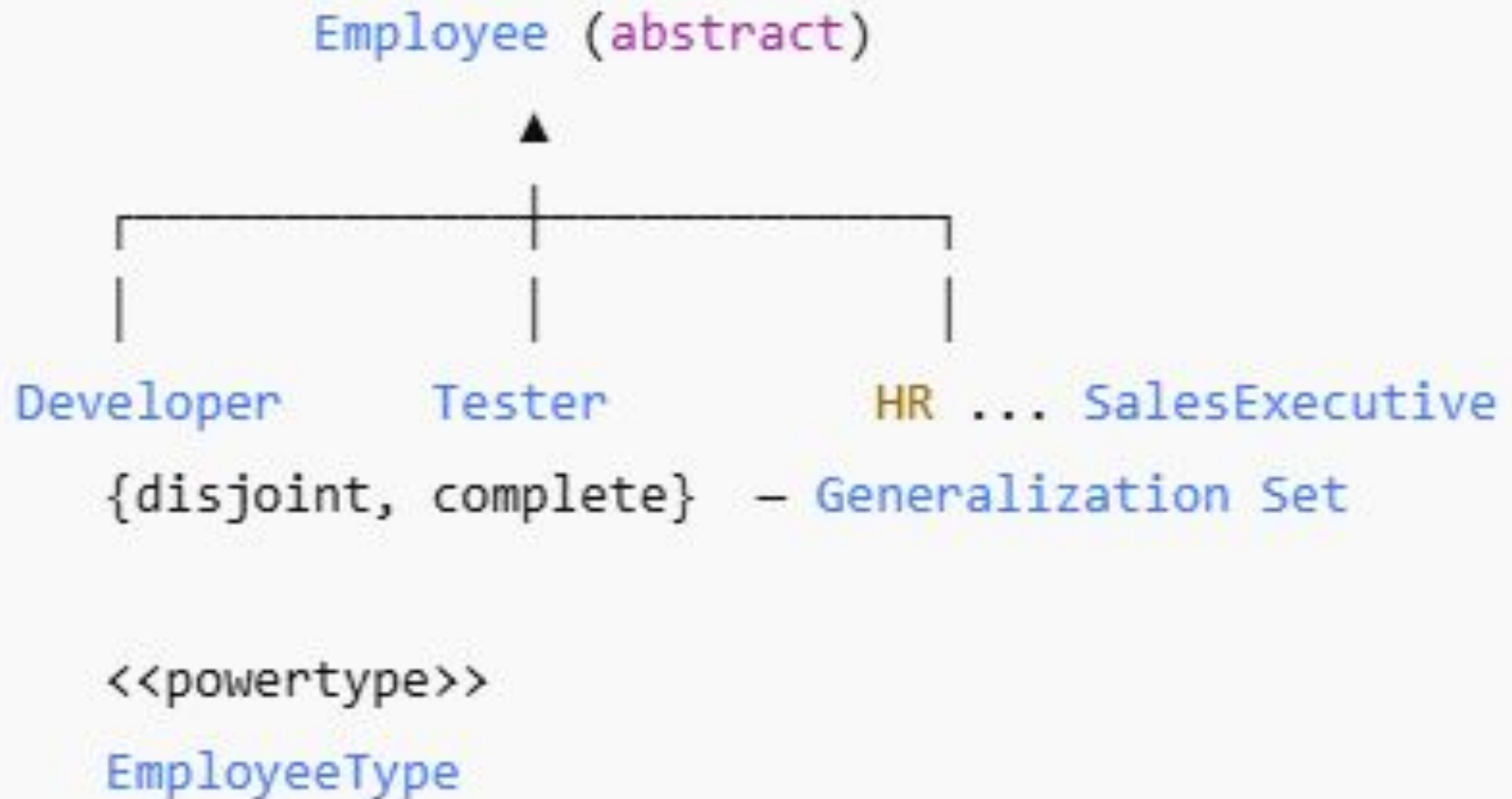
Powertype → has subclasses as instances.

Powertypes

A generalization set is connected to its powertype using a dashed line.

Written as <<powertype>>.

The powertype class sits beside or above the generalization set.



Generalization Set vs Powertype

Feature	Generalization Set	Powertype
Main purpose	Group generalization relationships	Represent subclasses/types as instances
Focus	Classification rules	Types of types
Specifies	Disjoint/overlapping and complete/incomplete constraints	A classifier whose instances correspond to subclasses
Example	Vehicle → Car, Bus, Truck	VehicleType → Car, Bus, Truck
Concerned with	How subclasses relate to superclass	Representing the subclasses themselves

THANK YOU